LLM-Mediated Natural Language Controls for Short-Video Recommender Systems: An Offline Study of Controllability

Vibhor Malik
M.Sc. in Applied Computing
British Columbia Institute of Technology
Supervisor: Borna Noureddin
COMP 9500 research project
Review draft: 14 October 2026, Vancouver time

***Abstract*: Experiment 1 shows that deterministic category enforcement works under the specified policy: TCP@10 rises from 11.6% to 79.2% and TCER@10 falls from 6.1% to zero, with enforcement supplied by construction rather than learned by the LLM. Llama parsed 120/120 templated inputs, 79/100 broader synthetic inputs and 2/10 adversarial cases. The broader-set score clears the proposal's 70% floor but misses its 85% target. These finite synthetic sets do not establish general language reliability. Explanations show no observed LLM advantage: all 10/10 generated fact selections equal the deterministic template. Steering costs predictive quality: boost requests reduce NDCG@10 by 45.7%, and the non-inferiority check is uninformative at this low baseline. The frozen backbone is below recency popularity: per-seed HR@1 is 0.0–0.46%, compared with 8.0% for the descriptive recency comparator, whose estimator uses pooled recent history. The study establishes bounded category control and its observed costs; it does not establish an LLM expressiveness advantage, more natural interaction, improved recommendation accuracy or useful quality preservation. Formal Experiment 2 awaits an approved replacement bank and independent annotation; exposed-practice results are reported separately.**

***Index Terms*: controllable recommendation, natural-language interfaces, sequential recommendation, offline evaluation, reproducibility**

I. Introduction

A recommendation feed can be well personalized while remaining difficult to steer. Item-level feedback communicates something about one video, whereas a request such as "show more of this category" specifies a broader change. An editable preference profile carries instructions across successive feed requests within a session. Explanations serve another purpose, exposing the information used to construct a recommendation. These functions are related, but their effects should not be conflated.

Mozilla's 2022 YouTube audit involved 22,722 volunteers and reported that Dislike prevented 12% and Not interested prevented 11% of unwanted recommendations relative to its no-feedback control [1]. This motivates investigating explicit control, but it is historical evidence about that platform, not a measurement of the current platform or a baseline for the present implementation.

Natural-language control creates two separate engineering questions. First, does a language model identify the requested operation and target? Second, does the ranking policy execute the resulting command correctly? A system can achieve perfect suppression of a category through a hard filter while using no language model at all. Conversely, an accurate parser can still feed an incorrect enforcement policy. Reporting only a final category proportion would hide this distinction.

The project therefore studies an explicit translation-and-enforcement pipeline. Its research question is: *How reliably can a local language model compile category-control requests, and what changes in category exposure and next-item recommendation accuracy result when those commands are applied to a frozen sequential recommender?* An additional engineering question concerns whether equivalent controls expressed through text and a persistent profile produce consistent state and rankings.

The contribution is an applied artifact and an auditable evaluation design, rather than a claim to introduce language-based recommendation or the first integrated control interface. The artifact exposes request parsing, structured validation, deterministic policy application and user-visible explanations as separate stages. The protocol prevents held-out outcome information from determining a request, preserves matched candidate sets, and treats users as the statistical units. Its experiments distinguish rule-based diagnostics from actual local-model execution. These choices make a modest result interpretable and reproducible.

No new human participants are involved in this study. Synthetic requests are experimental assignments, not observed user intentions. The benchmark can assess computational compliance and predictive trade-offs; it cannot demonstrate increased satisfaction, trust or agency. The supplied original proposal is an unsigned draft: neither research-path box is selected and the approval field is blank. This manuscript records an implemented study for review, without claiming an approved scope. The supplied form, a supplied transcript of the September 21, 2026 course email, and a detailed scope reconciliation accompany the artifact. The email transcript

specifies 135 hours over 15 weeks and a 6–10-page report; it does not approve this individual project's design.

II. Related Work

A. Interactive control and language-based profiles

Radlinski et al. motivate natural-language user representations as a way to make preferences inspectable and editable [2]. This is a research agenda, not evidence that the present implementation improves accuracy. User-controllable profiles also appear in the WWW Companion study by Woźniak et al. [3]; profile editing itself is therefore not a novelty claim here.

Niu et al. introduce TKGPT, a conversational short-video interface studied through interviews and a user study [4]. Its functions include articulating interests and understanding recommendations as well as control. This makes it inappropriate to characterize all prior work as isolated single-channel systems. Fabbri et al. also describe an integrated transparent and controllable short-video system with a preliminary user study [5]. The present work uses an offline computational design and does not reproduce these human-study outcomes.

Ramos et al. find that adding positive preferences to natural-language profiles raises target Coverage@10 without further fine-tuning after initial training [6]. This measures responsiveness, not higher predictive accuracy: on Amazon-MT, UPR(Mistral) RMSE is 0.941 versus matrix factorization's 0.925, where lower is better. Negative edits had only minor effects. Sanner et al. find language-based preferences competitive in a near cold-start movie setting [7], while Chen and Pu survey the earlier critiquing tradition [8]. Zhou et al. study language-based profiles through an encoder–decoder factorization approach [9]. Both establish textual profiles as a prior research direction. Here a profile is compiled into explicit category preferences around a fixed recommender, avoiding an unsupported comparison between differently trained recommendation architectures.

B. Controllable recommendation and the backbone

Lu et al. align an LLM using supervised learning and reinforcement learning, with labels augmented by a conventional recommender such as SASRec [10]. Their target-category-proportion metric measures category presence within the top-K output. The current system adopts this interpretable exposure measure, while keeping the recommendation model frozen and restricting the LLM to command translation.

Dai et al.'s COREC preprint uses learned control tokens for attribute-conditioned reranking [11]. Its hard-filter comparison illustrates why constraint satisfaction and recommendation utility must be reported together. This motivates explicit fill and quality metrics here, without assuming that an untuned parser matches a learned reranking model.

SASRec uses self-attention to predict subsequent items from interaction sequences [12]. The project implements a compact SASRec-style network in PyTorch and documents training choices rather than claiming exact reproduction of the original experiments. KuaiRec supplies dense short-video feedback and category metadata [13]. It supports a controlled offline testbed but does not expose responses to the interventions constructed here.

C. Evaluation validity

Ji et al. show that recommendation evaluation can leak future interactions when splits fail to respect a global timeline [14]. Chronological splitting only within each user does not prevent one user's later behavior from informing another user's earlier prediction. This benchmark instead establishes shared time boundaries before fitting the model.

A second risk is constructing a control request from the same held-out behavior later used to score the recommendation. Hiding held-out item identifiers does not remove this information path: a category extracted from those items still reveals an outcome-derived feature. The revised protocol avoids that path entirely. It also avoids treating display-only explanations as a ranking intervention, and avoids interpreting statistically nonsignificant differences as evidence of equivalence.

Zhang et al.'s 2026 vision paper proposes evaluating explicit contextual feedback with preference-alignment measures beyond historical ranking accuracy [15]. It motivates separating these constructs, but supplies no validation of this project's satisfaction bands or evidence of its accuracy gains.

III. Methodology

A. Scope, notation and information boundaries

Let $\mathcal{I}$ be the candidate item catalog and $G(i)$ the set of category identifiers assigned to item i. Each selected user u has an ordered training sequence H_u^{tr}, a later request-history sequence H_u^{rq}, and a final evaluation sequence H_u^{te}. Training accesses only the first sequence. At evaluation time the recommender receives the concatenation of training and request history, truncated to its maximum context length. Request construction can access these earlier sequences and static item metadata; it cannot access H_u^{te}.

The first eligible item in H_u^{te} is the next-item label y_u. A category command is $q = (o, c, \lambda)$, with operation o, category c and strength λ. The benchmark contains positive boosts and negative mutes. The interface also supports reset and clarification. A validated command is applied to the same frozen scores in every applicable configuration.

B. Dataset preparation

The preprocessing stage reads the official KuaiRec small matrix and item-category table. The upstream small matrix contains 1,411 users, 3,327 items and 4,676,570 interactions. These are source statistics; filtered experimental counts must be taken from the run manifest. KuaiRec has no explicit like label [16]. Accordingly, the preprocessing definition of a positive event is a watch ratio greater than 2.0, an engagement proxy rather than a direct preference statement.

Rows without timestamps are excluded rather than assigned guessed dates. Global timestamp quantiles divide all dated source events into nominal 70% training, 15% request-history

and 15% evaluation windows. The thresholds are then applied to positive events. A timestamp on a boundary enters the later window. Consequently, positive-event proportions after filtering need not equal the nominal split fractions.

The candidate catalog includes items with at least one positive event before the training cutoff. Later cold-item events outside that catalog are removed. A user must retain at least three training, two request-history and one evaluation event. Eligible users are ordered by a hash of a fixed cohort seed and their identifier, and a configured number is selected. This makes cohort selection independent of measured ranking outcomes, although requiring later observations still limits the target population to eligible active users. The frozen full-catalog policy permits already-seen items. A post-hoc check finds no repeated training items and no test item present in its user's earlier history in the retained cohort; repeated targets therefore do not justify this policy for these data. An unseen-only policy would require a separately identified experiment.

Categories remain the official opaque identifiers, rendered as `category_n`. No semantic topic names are inferred from numbers. Although newer official metadata provides richer information, a mapping must be checked before it is used. The current identifier-based task tests bounded command compilation and should not be described as understanding natural video topics. Repeated exposure to a previously seen item is allowed consistently across conditions.

C. Synthetic request construction

For each selected user, the generator forms the set of categories appearing in training or request history. For each operation independently, a hash of request seed, user identifier and operation selects a category from that set. The resulting assignment produces one boost and one mute request per user. It uses historical support, not a held-out category shift. The latest historical item in the selected category is the representative item for the feedback comparator.

The free-text channel uses controlled templates such as "Show more category_12" or "Do not show category_12". The profile channel conveys the same intent through preference wording. Template generation provides reproducibility and ensures a known intended operation, but it is narrower than spontaneous human language. Parser generalization is therefore evaluated separately on a dedicated utterance set with frozen labels.

The generator's independence is a software property that can be tested: changing evaluation labels must leave requests unchanged. It does not imply independence between historical behavior and future behavior, nor does it claim that the generated command represents what the user would actually request.

D. Sequential recommender

The independently implemented backbone uses item and positional embeddings, causal self-attention, pointwise feed-forward transformations and layer normalization. Padding positions are masked. The final hidden state produces a dot-product score for every candidate item. The default compact configuration has hidden dimension 32, two attention blocks, one attention head, context length 50 and dropout 0.2. The main study uses these dimensions with 20 fixed training epochs and seeds 42, 43 and 44, as frozen before test-outcome inspection.

Training breaks the available sequence into contiguous chunks with next-item targets. For each valid target, one negative item is sampled uniformly from items absent from that user's training-positive set. Request-history and final-test positives are not consulted for negative exclusion. Thus a sampled negative is an unobserved training item, not a verified dislike. The sampled logistic loss is

$$\mathcal{L} = \frac{1}{|T|} \sum_{t \in T} \left[\log(1 + e^{-s_t^+}) + \log(1 + e^{s_t^-}) \right]. \qquad (1)$$

The implementation uses Adam with learning rate 0.001, batch size 64 and gradient clipping at norm 5. Fixed-epoch runs measure a reproducible exploratory study; they do not establish convergence or state-of-the-art recommendation performance. Parameters are frozen across all five conditions within each training seed. Separate seeds assess limited training variability without creating additional independent users.

E. Command parser and editable profile

The language interface targets a local Llama 3.1 8B model through Ollama. A constrained response schema names the allowed operations, enumerates valid categories and bounds numeric values. Application validation remains necessary even when structured generation is requested. Unknown categories, nonfinite strengths and unsupported operations cannot enter the ranker.

A profile parser converts editable text into a bounded category-weight map. Positive weights denote boosts; a negative weight denotes a hard mute. A profile is session state, while a free-text command expresses the current update. Reloading the application starts a new session; durable storage across visits is not implemented. An explicit command takes precedence over a stored preference for the same category. This rule avoids accidentally applying the same request twice when channels are combined.

The system includes a transparent rule parser as a diagnostic comparator and demo mode. A rule result is never reported as an LLM result. Operational failures in local inference are retained with their error state. During an end-to-end evaluation, the failed channel applies no new control and the row remains in the denominator. This intention-to-treat treatment prevents a benchmark from improving merely by discarding difficult requests.

F. Shared control policy

Scores are normalized separately for each user using the catalog minimum and maximum. Equal scores map to zero. Let $\tilde{s}_u(i)$ denote the normalized score and $P_u(c)$ the effective profile or command weight. Items matching any muted category are removed. Remaining scores are

$$s'_u(i) = \tilde{s}_u(i) + \sum_{c \in G(i)} \max(P_u(c), 0). \qquad (2)$$

TABLE I
MATCHED EVALUATION CONFIGURATIONS

ID	Mechanism	Purpose
A	Simulated item-level feedback	Explicit reference policy
B	A plus explanation text	Ranking-invariance check
C	Parsed free-text control	Translation and enforcement
D	Parsed editable profile	Session preference path
E	Text, profile and explanations	Combined-path consistency

The default positive strength is 0.25. Identical intended controls share this strength and policy, regardless of interface channel. Descending adjusted score and ascending item identifier determine a stable ranking. A hard mute never pads an underfilled feed with forbidden items.

The item-feedback baseline differs deliberately in granularity: it boosts only the chosen historical exemplar or removes only that exemplar. It does not infer a category-wide intention from one item. This is an explicitly simulated comparator, not a reconstruction of the behavior of a commercial thumbs-up/down algorithm.

Table I defines the five conditions. They are matched configurations, not a complete 2^3 factorial design. Explanations alone cannot alter scores without a user or simulator responding to them, so B must exactly equal A. If C and D compile the same command and E deduplicates that command, their rankings can also coincide by construction. Such equality is consistency evidence, not proof that the interfaces have equal usability.

G. Explanations and stateful interaction

Explanation text is grounded in a finite set of verified statements: item identity, category membership, counts of matching categories in visible history, and the fact that model scores and explicit controls determine the displayed order. An optional local-model call selects one to three unchanged statements through a constrained schema. The application checks each selected statement against the allowed set before rendering it. This is LLM-mediated fact selection, not unrestricted explanation generation or proof of causal faithfulness.

Cached results include a signature of the item and its verified facts, together with model provenance. Changed verified facts produce a different signature; different histories with the same verified facts can share a signature. When no matching validated local-model cache entry exists, the interface explicitly labels a deterministic verified-template fallback. Therefore sample cache coverage must not be described as full-model explanation coverage for all user–item pairs. Neither path claims to recover the network's internal reasoning.

Offline one-turn comparisons cannot evaluate profile persistence. The application workflow therefore receives separate functional checks for setting preferences, editing them, changing users, resetting state and revisiting a feed. The same-category precedence rule is checked for boosts and mutes. These checks show what the software does; they cannot substitute for a study of whether people find the behavior understandable.

IV. EVALUATION DESIGN

A. Controllability and recommendation metrics

For returned list $L_{u,q}$ truncated to at most $K = 10$ items and target category c, let $m = |L_{u,q}|$ and

$$n_c(L) = \sum_{i \in L} \mathbf{1}[c \in G(i)]. \tag{3}$$

A multilabel item contributes at most once for the requested category. Positive-control target-category proportion is TCP@$K = n_c(L)/K$, and negative-control target-category exposure is TCER@$K = n_c(L)/K$. Higher TCP and lower TCER favor the requested intervention. These use the same arithmetic with different requested directions.

Fixed-K exposure can conceal empty outputs. The harness also reports conditional exposure $n_c(L)/m$ for nonempty lists, fill rate m/K, and a filled-compliance indicator. The latter requires all K slots to be filled and, for mute, zero target items; for boost it requires at least one target item. The boost indicator is a minimal presence check, not a guaranteed increase over the baseline. Empty-list conditional exposure is undefined, not zero.

Quality uses the first eligible held-out positive item: HR@1 $= \mathbf{1}[L_1 = y_u]$. The exploratory NDCG@K is zero if y_u is absent, and $1/\log_2(r+1)$ if it appears at rank r. With one label per user it measures next-item recovery, not utility for every candidate or every relevant future item. Commands may legitimately change that next-item outcome because the target records behavior before the proposed intervention.

B. Parser benchmark

The separate parser benchmark records exact full-command agreement against its fixed rubric: operation, category and strength must match the bounded command specification. Reset and clarification cases make unsupported or ambiguous requests visible rather than forcing an arbitrary guess. The final suite is retained separately from development examples as a versioned artifact. Exact case-insensitive strings do not overlap, but one development/test pair differs only by a final full stop; shared templates further limit independence. The annotation policy maps "less" and "fewer" to hard muting, treats conditional requests as unsupported, and asks for clarification when a neutral "show" request lacks direction. These restrictive choices must qualify the accuracy claim; they are fixed benchmark rules rather than established interpretations of human intent.

End-to-end correctness includes operational errors in the denominator. The report also separates successful inference, valid structured output and correct intent, so connection problems cannot be mistaken for semantic answers. Per-case raw output, parsed command, elapsed time and error state permit a failure analysis. Identical evaluation texts are cached by channel, category set and parser configuration. The number of user requests therefore differs from the number of unique

language inputs; ranking sample size must not be presented as independent parser-test sample size. A point-estimate target of 85% and a descriptive 95% Wilson interval are reported separately. For example, 85 correct cases in a set of 100 would have a lower bound near 76.7%; the target does not imply that the lower bound exceeds 85%.

The generated suite is not a probability sample of human requests. Shared wording templates can make items correlated and performance optimistic. The uncertainty interval summarizes a finite benchmark under a binomial working model; it does not establish population-wide natural-language understanding. Any prompt revision after test inspection creates a development result and requires a fresh reserved set for an unbiased final estimate.

C. Paired statistical analysis

All conditions operate on the same request and frozen score vector. The analysis averages matched outcomes within each user across available requests and fixed training seeds before inference. Thus n users with two requests and three seeds remain n sampling units rather than $6n$ independent users. Complete pairing is required, and rule and LLM runs are analyzed separately.

The primary contrasts are C, D and E versus A for positive TCP and negative TCER, a six-test family. B is a deterministic assertion rather than another hypothesis test. The harness reports mean paired changes and user-level percentile bootstrap intervals with 5,000 resamples. Its proposal-compatible Wilcoxon signed-rank tests use two-sided alternatives, Pratt zero handling, rounded differences and a normal approximation with ties. The all-zero case returns no evidence of change. These details matter because the metric is discrete and often tied [17].

Benjamini–Hochberg adjusted values accompany the six comparisons. Its assumptions include independence or suitable positive dependence [18]; the configurations are correlated, so these tests remain exploratory. A sign-flip sensitivity check is reported with its exchangeability or symmetry assumption. Neither a small p-value nor a bootstrap interval corrects proxy-intent bias, cohort selection or inadequate recommender training.

D. Exploratory non-inferiority

The original proposal's phrase "within 10%" is ambiguous. The frozen implementation chose an absolute HR@1 margin $\delta = 0.10$. This revision also reports the relative interpretation, $\delta = 0.10\overline{\mathrm{HR}}_A$, as a post-hoc sensitivity analysis. The baseline and paired difference both average boosts, mutes and seeds within user; using a boost-only baseline for that combined estimand would be inconsistent. For each control condition, paired quality change is $\Delta = \mathrm{HR}_{\text{condition}} - \mathrm{HR}_A$. Numerical support requires a lower confidence bound exceeding $-\delta$. The same Bonferroni-adjusted one-sided percentile bound accounts for three comparisons under each reading; reporting both is a sensitivity analysis, not a new confirmatory test or a way to select the favorable result. The relative margin uses the observed baseline as a fixed diagnostic threshold and does not propagate uncertainty in that threshold.

The absolute margin may be practically weak. If baseline HR@1 is less than 0.10, losing all baseline accuracy may still fall inside it. Degenerate empirical differences can also produce unrealistically narrow bootstrap intervals. Revised machine-readable results distinguish the mathematical criterion from informative quality support and flag a margin exceeding the baseline. Original results remain unchanged with their historical interpretation documented. Supervisor agreement can establish a meaningful margin for a future evaluation; it cannot retroactively prespecify one after these results. No claim of "no quality cost" follows from a numerical pass alone.

V. Implementation and Reproducibility

The software separates data preparation, model training, controls, evaluation and the web application. FastAPI exposes bounded endpoints for selecting users, fetching feeds, issuing commands and editing profiles. The React interface displays whether it is using real prepared data or a deterministic fixture, and whether the parser is rule based or an actual local model. Metadata cards represent items; no claim of playable KuaiRec video assets is made.

An unavailable data file cannot silently create real-data results. Fixture mode is useful for exercising state transitions and layout, but its outputs are tagged as demonstrations. Similarly, a local inference failure remains visible. The local-model endpoint is constrained to loopback, limiting accidental transfer to remote model services through an arbitrary URL.

A run records the dataset fingerprint, chronological boundaries, model checkpoint fingerprint, training seed, request seed, candidate policy, prompt or parser identity and result paths. Training metadata includes loss by epoch, configuration, dependency version, device and runtime. Request-level records retain intended controls, parser outcomes, top-K lists and ranking hashes. Summary tables and figures are derived from those records, rather than manually entered claims.

Verification checks information boundaries and meaningful failure modes: mutating test labels must not change requests; causal masking must exclude future sequence positions; save/load must preserve scores; mutes must never reintroduce excluded items; reset must clear preferences; combined controls must not double-count; malformed model output must remain an error. Browser acceptance covers a complete user workflow, state isolation and visible error states. These tests support artifact correctness but do not prove the research hypotheses.

The code can run with CPU resources for a small pilot. GPU availability affects speed, especially local-model latency, rather than the definition of correctness. Reproduction instructions are provided for Windows and Colab; full research execution on those platforms remains unverified. Any cloud run must record its own hardware and versions; portable code is not evidence of identical timing or bitwise-identical accelerator results.

TABLE II
POST-HOC UNCONTROLLED BASELINE DIAGNOSIS, 865 USERS

Model	HR@1 (%)	HR@10 (%)	NDCG@10
Uniform random (expected)	0.039	0.386	0.00175
Train-window popularity	0.116	0.578	0.00298
Request-window popularity	7.977	16.532	0.11866
SASRec, seed 42	0.000	4.509	0.01642
SASRec, seed 43	0.347	10.058	0.03627
SASRec, seed 44	0.462	11.098	0.04584

TABLE III
FROZEN SYNTHETIC PARSER TEST; CORRECT / CASES

Case group	Rule	Llama 3.1 8B
Positive	26/30	30/30
Negative	24/30	30/30
Reset	3/10	10/10
Ambiguous or unsupported	18/20	7/20
Adversarial	10/10	2/10
Total	81/100	79/100
Validation rejections	0	7

VI. Results and Evaluation

A. Executed cohort and training

The main prepared cohort includes all 865 eligible users and 2,593 candidates. The raw matrix contributes 4,676,570 records, of which 181,992 lack timestamps and are excluded. The catalog filter removes 43,708 post-cutoff positive cold-item events before user eligibility filtering. The final cohort retains 16,374 request-history and test positives (11,712 and 4,662), alongside 127,173 training positives. These counts refer to different filtering stages and are not a common-denominator retention fraction. The source archive's published MD5 and its recorded SHA-256 were verified; an independent preprocessing replay produced an identical prepared-data fingerprint.

An initial 256-user, three-epoch setup run measured execution cost without examining control or recommendation test outcomes. Its low cost allowed the full eligible-cohort scope and 20 fixed epochs to be frozen before outcome analysis. The three main checkpoints use seeds 42, 43 and 44. No model tuning followed the quality diagnosis below. This chronology is recorded in the main-run design and training artifacts rather than inferred from favorable results.

B. Weak backbone recommendation quality

The frozen backbone has very low next-item accuracy and highly concentrated first predictions. Table II reports a post-hoc descriptive diagnosis over all 865 users. The three seeds recover respectively zero, three and four next positive items at rank one. Training-only popularity recovers one; popularity pooled over the later request-history window recovers 69. Both popularity counts precede the test cutoff. SASRec uses each user's request history at inference, but its weights were fitted only on training-window interactions; recent popularity pools all users' later histories. This information-access difference prevents a controlled claim about model-class superiority. Nevertheless, the strong recent baseline exposes a substantial gap in the present recommendation setup. The random row is the exact expectation for a uniform permutation of 2,593 candidates with one relevant label, not a sampled run. Between 73.9% and 96.2% of users receive the same most-common SASRec first prediction within a seed; only two or three items occupy first position.

These outcomes do not establish a competitive recommender. They place the secondary quality metric near a floor and make the absolute 0.10 non-inferiority margin uninformative about practical preservation of recommendation quality. An apparent pass under that margin would permit almost all of the already small baseline accuracy to disappear. The remaining experiments therefore serve as evidence about command parsing and mechanical control behavior around this fixed backbone, not evidence that useful personalization is preserved.

C. Frozen local-model parser benchmark

Llama 3.1 8B, quantized as Q4_K_M under Ollama 0.34.4, achieves 79/100 exact full-command matches. The descriptive 95% Wilson interval is [70.02%, 85.83%]. **The 85% point-estimate target from Objective 3 and Expected Outcomes of the original proposal is not met. The result clears the proposal's 70% floor: its "control channels degraded" criterion is accuracy below 70%.** These are proposal criteria, not evidence of signed approval. The interval includes 85%; target status is descriptive, not a rejection of a population-accuracy hypothesis. The transparent rule comparator obtains 81/100, with interval [72.22%, 87.49%]. The two-point descriptive difference is not evidence that either approach is generally superior; no paired superiority analysis was prespecified.

All 70 supported boost, mute and reset cases are correct under the fixed rubric. Only nine of the 30 cases requiring clarification are correct. Fourteen cases return a valid but rubric-incorrect command; these are parser outputs, not evidence of fourteen executed application-state changes. Seven cases are rejected by the grounding validator; these are recorded as operational errors but are not network failures. For example, "Do not hide category_10" incorrectly becomes a mute, and a bare category name becomes a boost. The 2/10 adversarial accuracy exposes susceptibility to embedded instructions. Allowlist and grounding validation restrict executable values but cannot ensure that a valid command reflects the user's intended instruction. Unknown or ungrounded targets are rejected instead of producing clarification. Such rejection prevents an invalid action while still failing the interface contract.

The suite uses synthetic, single-author gold labels and a restrictive interpretation policy. Its four allowed identifiers are category_0, category_1, category_2 and category_10, rather than the full 31-category study catalog. Its group balance and exact accuracy do not estimate the distribution of ordinary user requests. The development suite is excluded from these

TABLE IV
ACTUAL LLAMA CONTROL RESULTS; MEANS OVER 865 USERS

Metric	A/B	C/D/E
Positive TCP@10 (%)	11.615	79.222
Negative TCER@10 (%)	6.112	0.000
Fill rate, both operations (%)	100.000	100.000
Positive HR@1 (%)	0.231	0.539
Negative HR@1 (%)	0.270	0.347
Positive NDCG@10	0.03012	0.01637
Negative NDCG@10	0.03370	0.03526

results. Following an interrupted worker, the run resumes from 96 already recorded outcomes and completes the remaining four cases without changing prompts, labels or model identity. Original records are retained. Cached evaluation durations are not measurements of model inference speed.

D. Matched local-model control evaluation

The canonical end-to-end task contains 120 unique language inputs from four templates: 60 text commands and 60 equivalent profile statements, covering 30 assigned categories per operation and 31 distinct categories across operations in each channel. All 120 produce the intended controls without recorded errors. Their wording patterns were used during prompt development; this is not a held-out language-generalization test. Results are cached by input and provenance for reuse across users and training seeds. The ranking study therefore does not contain 5,190 independent language tests.

The completed actual-model ranking run evaluates 1,730 assigned requests per seed and 25,950 condition–request rows overall. Table IV shows means over 865 users after averaging training seeds. There are no canonical parser errors or clarification outputs. All lists contain ten items. A and B are rank-identical, and C, D and E are rank-identical under the matched compiled commands.

For C versus A, the mean paired positive change is 67.607 percentage points, with a 95% user-bootstrap interval of [65.788, 69.438]. The negative change is -6.112 points, with interval [-7.056, -5.241]. D and E yield identical contrasts. The corresponding BH-adjusted Wilcoxon values are approximately 1.35×10^{-142} for boosts and 1.05×10^{-65} for mutes. They quantify consistent changes under the assigned task, not an LLM advantage over a direct structured command.

The proposal's primary directional criterion (higher TCP and lower TCER for at least one of B–E versus A under Wilcoxon tests with BH adjustment) is numerically supported by C/D/E. This must be read together with the policy construction: correct hard mutes guarantee zero target exposure, and category-wide boosts directly favor the metric against a single-item comparator. The identical rule-parser result attributes the measured effect to enforcement, not language-model intelligence. A positive boost does not mathematically guarantee a strict gain for every possible catalog or score vector, so the reported test should not be described as universally incapable of failing. In this cohort, saturation and comparator granularity sharply limit its evidential strength.

The post-hoc saturation check finds TCP@10$=1$ for 373, 417 and 325 of 865 C-condition boost requests in seeds 42, 43 and 44 respectively; 287 users saturate in all three seeds. A can change at most one target item, hence at most 0.1 TCP relative to the uncontrolled list, whereas a category policy can affect many items. Category sizes range from one to 632 across the catalog. Category-level exposure and size records accompany the supplemental analysis. These observations expose a policy-granularity confound and do not establish a language-interface effect.

The quality measures do not move uniformly. For positive requests, next-item NDCG@10 decreases from 0.03012 under A to 0.01637 under C, a relative decline of approximately 45.7%. The HR@1 point estimates in the quality table are descriptive; the pooled interval below addresses a different estimand and does not establish a boost-specific accuracy improvement. Negative-request NDCG@10 changes from 0.03370 to 0.03526. The positive-request decrease measures poorer alignment with a held-out historical next item, not demonstrated user dissatisfaction: the assigned control changes the objective, while the recorded future behavior was collected without that intervention. The disagreement reinforces the danger of relying on the very sparse top-1 metric or interpreting an absolute-margin non-inferiority pass as preservation of useful quality.

A post-hoc paired boost C–A analysis gives mean NDCG@10 change -0.01375, 95% user-bootstrap interval [-0.02086, -0.00655], and two-sided Wilcoxon–Pratt $p = 3.23 \times 10^{-10}$, with 108 of 865 users having nonzero differences. This unadjusted exploratory secondary contrast lies outside the unchanged primary BH family; it was added after inspection and is not a prespecified confirmatory result.

Averaging both operation types, the paired HR@1 difference is 0.193 percentage points, with interval [-0.058, 0.501]. Only eight of 865 users have nonzero paired HR@1 differences. The baseline for this combined estimand is 0.2505%, rather than the boost-only 0.2312%. The Bonferroni-adjusted lower bound is -0.07707 percentage points. All three conditions numerically pass the absolute -10-point threshold, but fail the relative threshold of -0.02505 points (10% of that baseline). Thus absolute non-inferiority is an exploratory, practically vacuous numerical pass; relative non-inferiority is not supported. Neither establishes preserved useful quality, and the two readings must remain visible together.

E. Rule comparator and artifact verification

The separately executed rule diagnostic produces exactly the same full-ranking hashes and aggregate metrics as the actual-model canonical run in the recorded environment. It therefore isolates the shared policy's mechanical effect. The LLM adds a translation path, but these canonical results provide no evidence that it outperforms deterministic control. Its broader language limitations remain visible in Table III. Packaged checkpoints anchor the original experiment. An external audit reported matching top-ten lists and metrics with those checkpoints under a different PyTorch version, but

differences in full-catalog ranking hashes and newly trained weights. The later October numerical audit reproduced all 25,950 top-ten lists, while finding 2,009 full-ranking hash differences and 5,190 request-score hash differences whose cause remains unresolved. Dependency pins support reproduction; they do not guarantee bitwise equality across environments. A retraining run is a separately identified replication, not a replacement for the recorded models.

Recorded automated verification reconstructs all prepared user sequences from the source archive, checks the three checkpoint hashes, and recomputes all 25,950 actual-model result rows and paired summaries. The original release passed fifteen browser checks; the recorded revision passed seventeen, including reset, user isolation, cached-explanation labels, keyboard submission, visible inference failure and mobile layout. Those checks exercised the rule diagnostic and a deliberately unavailable inference endpoint. A separate recorded revision run passed 21 API checks through three fresh, uncached local-model calls for boost, mute and an edited profile; it also verified isolation and reset. These are historical Linux execution records, not fresh inference in the present delivery environment or Windows or Colab acceptance. The separate explanation demonstration generates ten actual local-model fact selections for the first user's ten baseline items, with zero recorded errors. Every selected statement belongs to that item's verified fact set. All ten outputs exactly equal the deterministic template's first three allowed statements, so this sample shows no observed explanation-selection advantage from the LLM. This is a limited cache sample, not a ten-user study or a model-generated explanation for every evaluated recommendation. Main ranking experiments use verified policy-template explanations; the optional local-model cache is demonstrated separately. No human explanation-quality or causal-faithfulness outcome is measured.

B–A equality is expected from the design, and C/D/E equality may follow from matched commands. Neither establishes interface equivalence. Zero residual exposure after correct mute parsing is an enforcement property. Empirical interpretation requires parser correctness, available list fill and the weak-backbone caveat above.

VII. Discussion

A. What this experiment can establish

The pipeline makes three claims independently testable. A parser either matches the intended structured command or it does not. A policy either applies the validated command consistently or it does not. A particular held-out ranking metric changes by a measured amount. Separating these claims helps distinguish a useful language interface from a deterministic filter's expected success.

The design also makes negative findings informative. A low parser score identifies a language-interface limitation even if oracle commands yield perfect compliance. Identical C and D outputs validate common semantics but leave interface preference unresolved. A quality loss after a correct control can reflect the intended change in recommendations, an overstrong policy, a weak backbone, or mismatch between the historical next-item label and the requested intent. The benchmark should not select the most flattering explanation without further evidence.

B. Threats to validity

Construct validity. Opaque category labels and templated requests substantially narrow the language task. A historical category is not equivalent to a genuine current desire. The single-item feedback comparator is weaker in information granularity than a category instruction, and so it cannot attribute gains specifically to an LLM. Direct structured commands and rule parsing are diagnostic reference points.

Internal validity. Shared chronological boundaries and test-label mutation checks reduce specific leakage risks, but the experiment is retrospective. Time boundaries use dataset-wide timestamp quantiles, and eligibility depends on adequate observations in all windows. Recorded outcomes do not simulate changed exposure or feedback after a command. Incorrect metadata or inconsistent filtering could still invalidate a run, making saved provenance and schema checks necessary.

Statistical validity. Requests and seeds from one user are dependent. Aggregating at the user level addresses pseudoreplication but does not make a selected active-user cohort representative of all platform users. The three executed training seeds provide a small stability check. Ties, zero differences, shared templates and degenerate bootstrap distributions restrict interpretation of inferential summaries.

External validity. KuaiRec comes from a specific platform and period. Positive watch ratio is a proxy and its threshold can alter the cohort. A CPU study with a compact untuned backbone is not evidence about a production-scale recommender. No human experiment tests comprehension, perceived agency or real-world satisfaction. The interface uses item metadata rather than original streaming video.

C. Ethics, authorship and research transparency

The study uses a public dataset and conducts no new participant recruitment. Public availability does not justify attempting to reidentify users, and the artifact need not display raw personal attributes. The implementation exposes only the information required for the demo. Dataset and model licenses should accompany redistribution instructions rather than being replaced by a blanket claim that all components have the same license.

The package used substantial AI assistance for research synthesis, code, testing, documents, model training, inference and statistical analysis. The student remains responsible for understanding and reproducing the work, checking course policy and acknowledging assistance. The prospective 135-hour plan certifies no completed student work. Exposed test data cannot become untouched through a later freeze. Effort, supervisory decisions and revisions require contemporaneous records; generated records cannot substitute for meetings, signatures or presentation.

VIII. Experiment 2: Prespecified Expressiveness Study

A. Original design and pending formal status

The original design covers compound, graded, nonzero-floor and same-item conditional intents, with 48 development and 100 originally reserved questions across 31 opaque categories. Development v1 recorded rule 48/48 and Llama 24/48, including ten Llama operational errors; development v2 recorded 48/48 for both after permitted development changes. These are development results. The 100 worked questions and AI answers have since been exposed as practice. Formal evaluation therefore needs prospective supervisor agreement on a replacement unseen bank and a valid independent annotation route; the original September 29 parser commitment remains preserved.

The original procedure requires genuine independent labels, agreement before blinded adjudication, a full freeze and verified model provenance. It defines H1 as an advantage in induced satisfaction against both item feedback and rules, H2 as intended movement against item feedback, and H3 as descriptive quality cost. Rules and Llama share the ranker; one historical item reaction is a diagnostic comparator, not equal semantic information or effort. No formal hypothesis decision follows from the exposed practice run.

The protocol fixes operational count bands for compound, graded, floor and intersection requests; these conventions do not establish human satisfaction. Success requires positive intended movement. SHA-256 assignment uses training and request history; relevance labels enter only after ranking. Analysis averages seeds within user, retains 5,000 user bootstrap resamples and an eleven-test BH family, and tunes no band or weight on outcomes.

B. Exposed practice diagnostic on 14 October 2026

The student authorized a local download of the original `llama3.1:8b` model from the official Ollama library. Its Q4_K_M digest is `46e0c10c039e` (full digest in the execution record). Ollama was version 0.34.0, compared with historical 0.34.4. A separate protocol was saved before inference. The existing E2 prompt, schema, validation, rules and ranking implementation were reused unchanged, with temperature 0, seed 42, context 4096, output limit 768, six threads and a 600-second request timeout. There was no retry, tuning or model substitution in the completed run.

The 100 exact questions and labels were copied from the delivered worked-practice CSV. Those labels are AI-generated examples, not independent human gold. Class assignments follow their directive shapes. Llama agreed with 73/100 references and rules with 18/100; Llama had 24 validation rejections, retained in the 100-case denominator (23 conditional-grounding checks and one invalid floor). These scores measure agreement with an exposed AI reference, not unseen-test generalization or human-label accuracy.

Using the three frozen recommender seeds, the diagnostic produced 41,508 ranking rows for 3,459 assigned user/class requests and 865 users. Assignment used training and request history; final behavioral labels were accessed only for relevance scoring. User-averaged induced success was 0.0392 for the item-feedback comparator, 0.0840 for rules and 0.2381 for Llama. Practice NDCG@10 was 0.02461 for Llama versus 0.03128 for item feedback, so greater control movement did not imply greater recommendation quality. These are descriptive practice results under the existing operational satisfaction bands. Full score vectors, predictions, raw model responses, failures and all exclusions are retained. A separate standard-library audit recomputed every row's control/relevance metrics and pooled means without discrepancies. The same AI agent performed this audit; bootstrap and signed-rank code were not independently reimplemented.

All 288 E1 file hashes and the original parser commitment remained unchanged. Formal H1, H2 and H3 remain pending prospective supervisor approval of a replacement unseen bank, valid independent annotation, agreement, blinded adjudication and full freeze.

IX. Conclusion

Deterministic category enforcement works under the Experiment 1 policy: TCP@10 increases from 11.6% to 79.2% and TCER@10 decreases from 6.1% to zero, reflecting control built into the ranker. Llama parsed 120/120 templated inputs, 79/100 broader synthetic inputs and 2/10 adversarial cases. The broader-set score clears the 70% floor but misses the 85% target. These finite synthetic sets do not establish general language reliability. Explanations show no observed LLM advantage because all 10/10 outputs equal the template. Steering costs predictive quality: boost NDCG@10 falls 45.7%, while the non-inferiority check is uninformative at this baseline. The backbone is below recency popularity, with per-seed HR@1 of 0.0–0.46% versus 8.0% for the descriptive comparator. The study establishes bounded category control and its observed costs; it does not establish natural-language expressiveness, naturalness, an accuracy improvement or useful quality preservation.

References

[1] B. Ricks and J. McCrosky, "Does this button work? investigating YouTube's ineffective user controls," Mozilla Foundation, Tech. Rep., Sep. 2022. [Online]. Available: https://www.mozillafoundation.org/en/research/library/user-controls/report/

[2] F. Radlinski, K. Balog, F. Diaz, L. Dixon, and B. Wedin, "On natural language user profiles for transparent and scrutable recommendation," in *Proceedings of the 45th International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2022. [Online]. Available: https://arxiv.org/abs/2205.09403

[3] S. Woźniak, J. Duszenko, J. Kocoń, and P. Kazienko, "Improving LLM-based recommender systems with user-controllable profiles," in *Companion Proceedings of the ACM on Web Conference 2025*. Association for Computing Machinery, 2025, pp. 2102–2111. [Online]. Available: https://doi.org/10.1145/3701716.3717734

[4] S. Niu, D. Vishnuvardhan, and V. S. R. Punnam, "Chat with the 'for you' algorithm: An LLM-enhanced chatbot for controlling video recommendation flow," in *Proceedings of the 7th ACM Conference on Conversational User Interfaces*, 2025, pp. 1–16. [Online]. Available: https://commons.clarku.edu/faculty_computer_sciences/242/

[5] M. Fabbri, J. Jia, P. Jerez Arnau, and W. Wörndl, "Feeding the (short-video) feed: A design proposal for user control of social media recommender systems under the Digital Services Act," *International Journal of Human-Computer Studies*, vol. 214, p. 103811, 2026. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1071581926000868

[6] J. Ramos, H. A. Rahmani, X. Wang, X. Fu, and A. Lipani, "Transparent and scrutable recommendations using natural language user profiles," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 2024, pp. 13 971–13 984. [Online]. Available: https://aclanthology.org/2024.acl-long.753/

[7] S. Sanner, K. Balog, F. Radlinski, B. Wedin, and L. Dixon, "Large language models are competitive near cold-start recommenders for language- and item-based preferences," in *Proceedings of the 17th ACM Conference on Recommender Systems*, 2023. [Online]. Available: https://arxiv.org/abs/2307.14225

[8] L. Chen and P. Pu, "Critiquing-based recommenders: survey and emerging trends," *User Modeling and User-Adapted Interaction*, vol. 22, pp. 125–150, 2012.

[9] J. Zhou, Y. Dai, and T. Joachims, "Language-based user profiles for recommendation," 2024. [Online]. Available: https://arxiv.org/abs/2402.15623

[10] W. Lu, J. Lian, W. Zhang, G. Li, M. Zhou, H. Liao, and X. Xie, "Aligning large language models for controllable recommendations," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 2024, pp. 8159–8172. [Online]. Available: https://aclanthology.org/2024.acl-long.443/

[11] W. Dai, W. Xu, P. Wang, and D. N. Metaxas, "Token-controlled re-ranking for sequential recommendation via LLMs," 2025, preprint, version 1, submitted 22 November 2025. [Online]. Available: https://arxiv.org/abs/2511.17913

[12] W.-C. Kang and J. McAuley, "Self-attentive sequential recommendation," in *2018 IEEE International Conference on Data Mining (ICDM)*, 2018, pp. 197–206. [Online]. Available: https://arxiv.org/abs/1808.09781

[13] C. Gao, S. Li, W. Lei, J. Chen, B. Li, P. Jiang, X. He, J. Mao, and T.-S. Chua, "KuaiRec: A fully-observed dataset and insights for evaluating recommender systems," in *Proceedings of the 31st ACM International Conference on Information and Knowledge Management*, 2022, pp. 540–550. [Online]. Available: https://kuairec.com/

[14] Y. Ji, A. Sun, J. Zhang, and C. Li, "A critical study on data leakage in recommender system offline evaluation," *ACM Transactions on Information Systems*, vol. 41, no. 3, pp. 1–27, 2023. [Online]. Available: https://arxiv.org/abs/2010.11060

[15] W. Zhang, W. Yang, Y. Cui, Z. Guo, H. Hu, L. Yang, H. P. Zou, Q. Wang, H. Zeng, J. Liu, Y. Xia, and P. S. Yu, "Towards user preference alignment in LLM recommendation via explicit context feedback," 2026, vision paper, preprint version 1. [Online]. Available: https://arxiv.org/html/2605.29141v1

[16] KuaiRec Authors, "KuaiRec official dataset documentation," 2026, accessed 28 September 2026; dataset is originally from 2022. [Online]. Available: https://github.com/chongminggao/KuaiRec

[17] SciPy Developers, "scipy.stats.wilcoxon documentation," 2026, accessed 28 September 2026; record the actually installed SciPy version in each experiment. [Online]. Available: https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.wilcoxon.html

[18] SciPy Developers, "scipy.stats.false_discovery_control documentation," 2026, accessed 28 September 2026. [Online]. Available: https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.false_discovery_control.html